



Universidade Federal do Rio Grande do Norte
Departamento de Informática e Matemática Aplicada
DIM0124 - Programação Concorrente

Relatório de Desenvolvimento de Aplicação Concorrente

Algoritmo K-Means

Matrícula: 20210081823

Nome: Rary Emanuel Gonçalves Coringa

Contents

1	Introdução	2
1.1	Descrição do Algoritmo	2
1.2	Descrição da Linguagem Erlang	2
2	Configuração dos Experimentos	3
2.1	Configurações do Computador	3
2.1.1	Hardware	3
2.1.2	Software	3
2.2	Configurações de Testes	3
2.2.1	Microbenchmark (JMH)	3
2.2.2	Macrobenchmark (JMeter)	4
2.2.3	Stress Test (JCStress)	5
2.2.4	Profile (JFR)	6
2.2.5	Microbenchmark em Erlang	6
2.2.6	Profile em Erlang	6
3	Implementação Serial	7
3.1	Implementação em Java	7
3.1.1	Descrição	7
3.1.2	Resultados	9
3.2	Implementação em Erlang	11
3.3	Discussão	11
4	Implementação Concorrente	12
4.1	Implementação em Java	12
4.1.1	Descrição	12
4.1.2	Resultados	14
4.2	Discussão	17
5	Implementação Concorrente — Dados em Memória	19
5.1	Implementação em Java	19
5.1.1	Descrição	19
5.1.2	Resultados	19
5.2	Implementação em Erlang	21
5.2.1	Descrição	21
5.2.2	Resultados	21
5.3	Discussão	22
6	Discussão Final	24
6.1	Comparativo Geral das Abordagens	24
6.2	Pontos Positivos e Negativos de Cada Abordagem	25
6.3	Lições Aprendidas	26
7	Repositórios	27

1 Introdução

1.1 Descrição do Algoritmo

O K-Means é um algoritmo de agrupamento não supervisionado que particiona um conjunto de dados em k grupos (clusters), de forma que cada ponto pertença ao cluster cujo centróide é o mais próximo. O algoritmo opera em iterações: na etapa de **atribuição**, cada ponto é associado ao centróide mais próximo segundo a distância euclidiana; na etapa de **atualização**, cada centróide é recalculado como a média aritmética de todos os pontos a ele atribuídos. Esse ciclo se repete até que os centróides estabilizem (deslocamento máximo abaixo de uma tolerância) ou o número máximo de iterações seja atingido.

A complexidade por iteração é $O(N \cdot k \cdot d)$, onde N é o número de pontos, k o número de clusters e d a dimensionalidade dos dados. Por percorrer todos os pontos a cada iteração, o algoritmo é naturalmente paralelizável na etapa de atribuição, tornando-o um caso de estudo relevante para comparação entre estratégias de concorrência.

1.2 Descrição da Linguagem Erlang

Erlang é uma linguagem funcional criada pela Ericsson, projetada para sistemas distribuídos e tolerantes a falhas. Executa sobre a máquina virtual BEAM, que provê concorrência nativa por meio de processos leves e isolados.

Modelo de programação. Utiliza passagem de mensagens (modelo de atores). Processos não compartilham memória e se comunicam exclusivamente via caixas postais, eliminando por construção os problemas de acesso concorrente a estado compartilhado.

Modelo de memória e visibilidade. Cada processo possui heap privado. Mensagens são copiadas entre heaps ao ser enviadas, o que garante isolamento total. Por não haver memória compartilhada mutável, questões de visibilidade — como barreiras de memória e `volatile` do Java — não se aplicam.

Controle de região crítica. Não existem mutexes ou semáforos. O acesso a recursos exclusivos é feito encapsulando-os em um processo servidor que atende requisições sequencialmente — padrão formalizado pelo `gen_server` OTP.

Coleta de lixo. O GC opera por processo, de forma independente, sem pausas globais (*stop-the-world*). Processos de vida curta têm seus heaps simplesmente descartados ao terminar.

JIT. A partir do OTP 24, a BEAM inclui um compilador JIT (AsmJit) que gera código nativo para x86-64 e AArch64 no carregamento dos módulos, com ganhos típicos de 20–40% em relação ao bytecode interpretado.

2 Configuração dos Experimentos

2.1 Configurações do Computador

Esta seção descreve o ambiente de hardware e software utilizado na execução dos experimentos. O conhecimento dessas configurações é essencial para a reprodutibilidade dos resultados e para a correta interpretação das métricas coletadas.

2.1.1 Hardware

CPU: O processador utilizado é o **Intel Core i5-13450HX** (13ª geração), com arquitetura híbrida composta por 6 P-cores (Performance, com Hyper-Threading) e 4 E-cores (Efficiency, sem Hyper-Threading), totalizando 10 núcleos físicos e 16 threads lógicas. A frequência máxima é de 4,6 GHz. As memórias cache são: L1d 416 KiB, L1i 448 KiB, L2 9,5 MiB e L3 20 MiB.

RAM: A memória RAM total é de **16 GB DDR5** em configuração dual-channel (2 × 8 GB), com velocidade máxima de 5600 MT/s e frequência configurada em 4800 MT/s.

Disco: O armazenamento em disco utilizado é um **SSD NVMe** com capacidade de 512,1 GB.

2.1.2 Software

Sistema Operacional: **Ubuntu 24.04.4 LTS**, kernel 6.17.0-22-generic.

Java: **OpenJDK 25.0.1** - Temurin-25.0.1+8 (build 25.0.1+8-LTS).

Erlang: **Erlang/OTP 25**, emulador BEAM versão 13.2.2.5, com JIT AsmJit habilitado.

2.2 Configurações de Testes

2.2.1 Microbenchmark (JMH)

Os microbenchmarks foram implementados com **JMH 1.37** sobre **JDK 25.0.1**, com modo **AverageTime**, 1 iteração de aquecimento de 10 s, 3 iterações de medição de 10 s e 1 fork por benchmark. O estado do dataset é inicializado uma vez por trial (**@Setup(Level.Trial)**) e os vetores de hotspot usam semente fixa (**seed = 42**) para reprodutibilidade.

ID	Método	Objetivo
B1	b1_fullRun	Execução completa ($k=10$, maxIter=20, tol= 10^{-6}) — número primário do relatório
B2	b2_perIterationCost	Execução curta ($k=10$, maxIter=5, tol=0) — isola custo por iteração
B3	b3_closestCentroid	Chamada isolada de closestCentroid — ponto quente do laço interno
B4	b4_squaredDistance	Chamada isolada de squaredDistance — baseline aritmético

Table 1: Casos de benchmark JMH definidos em KMeansBenchmark.

A Figura 1 apresenta a classe de benchmark com as anotações de configuração e os quatro métodos medidos.

```
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.SECONDS)
@Warmup(iterations = 1, time = 10)
@Measurement(iterations = 3, time = 10)
@Fork(1)
public class KMeansBenchmark {

    @Benchmark
    public KMeansResult b1_fullRun(DatasetState s) throws IOException {
        return KMeans.cluster(s.path, s.dataset, 10, 20, 1e-6);
    }

    @Benchmark
    public KMeansResult b2_perIterationCost(DatasetState s) throws IOException {
        return KMeans.cluster(s.path, s.dataset, 10, 5, 0);
    }

    @Benchmark
    public int b3_closestCentroid(HotspotState s) {
        return KMeans.closestCentroid(s.point, s.centroids);
    }

    @Benchmark
    public double b4_squaredDistance(HotspotState s) {
        return KMeans.squaredDistance(s.left, s.right);
    }
}
```

Figure 1: Configuração JMH e casos de benchmark em KMeansBenchmark.

2.2.2 Macrobenchmark (JMeter)

Os macrobenchmarks foram implementados com **Apache JMeter 5.6.3** utilizando o mecanismo de *Java Sampler* (`AbstractJavaSamplerClient`). A classe `KMeansSampler` executa `KMeans.cluster()` diretamente na JVM do JMeter — sem servidor HTTP intermediário — permitindo que o coletor de GC configurado na inicialização do JMeter se aplique integralmente ao algoritmo.

O GC é selecionado pela variável de ambiente `GC_ALGO` passada ao script de inicialização do JMeter. Foram avaliados três coletores:

- `-XX:+UseG1GC` — G1GC (coletor padrão do JDK 25);
- `-XX:+UseZGC` — ZGC (coletor de baixa pausa);
- `-XX:+UseParallelGC` — ParallelGC (coletor paralelo de alta vazão).

A matriz de testes combina os três coletores com cinco contagens de threads, totalizando 15 execuções independentes:

Parâmetro	Valor
Coletores de GC	G1GC, ZGC, ParallelGC
Threads JMeter	1, 2, 4, 8, 16
Duração por execução	120 s
Ramp-up	30 s
k	10
maxIter	3
tol	0,0
Métrica primária	Throughput (execuções/min)

Table 2: Configuração do macrobenchmark JMeter.

2.2.3 Stress Test (JCStress)

Os testes de condição de corrida foram implementados com **JCStress 0.16** e executados sobre **JDK 25.0.1**. O JCStress instancia múltiplos processos JVM (*forks*) e varia a combinação de compiladores (Interpreter, C1, C2) e a afinidade de CPUs entre forks, maximizando a cobertura de interleavings possíveis.

Parâmetro	Valor
Ferramenta	JCStress 0.16
Preset	<code>default</code>
Forks por teste	6 (1 normal + 5 stress)
Iterações por fork	5
Tempo por iteração	1.000 ms
CPUs utilizadas	16

Table 3: Configuração do stress test JCStress.

Foram definidos quatro testes, cada um associado a uma região crítica do algoritmo:

ID	Classe	Região crítica
JCS1	<code>JCS1_ClosestCentroidTest</code>	Leitura concorrente do snapshot de centróides
JCS2	<code>JCS2_SquaredDistanceTest</code>	Leitura concorrente de <code>double[]</code> compartilhado
JCS3	<code>JCS3_LocalCountAccumulationTest</code>	<code>counts[0]++</code> em array compartilhado (abordagem ingênua)
JCS4	<code>JCS4_LocalSumAccumulationTest</code>	<code>sums[0] += 1.0</code> em array compartilhado (abordagem ingênua)

Table 4: Testes JCStress e regiões críticas associadas.

2.2.4 Profile (JFR)

O profile foi capturado com **JFR (Java Flight Recorder)**, ferramenta embutida no JDK 25 — sem necessidade de instalação adicional. A gravação foi iniciada via flag de JVM na execução do algoritmo e analisada exclusivamente com a ferramenta de linha de comando `jfr` do JDK, sem uso do JMC (*Java Mission Control*).

A configuração `settings=profile` habilita amostragem de CPU em alta resolução (intervalo de 10 ms), adequada para identificar hot methods. A flag `dumponexit=true` garante que o arquivo `.jfr` seja gravado ao final da execução mesmo sem intervenção manual.

Parâmetro	Valor
Ferramenta	JFR (JDK 25 built-in)
Configuração	<code>settings=profile</code>
Captura	<code>dumponexit=true</code>
Análise	<code>jfr print</code> (CLI)
GC	G1GC
k	10
<code>maxIter</code>	3
Dataset	≈ 1 GB

Table 5: Configuração do profile JFR.

Os eventos extraídos com `jfr print -events` cobrem quatro áreas:

- `jdk.ExecutionSample` — amostras de CPU por método (hot methods);
- `jdk.GarbageCollection`, `jdk.GCHeapSummary` — pausas e uso de heap ao longo da execução;
- `jdk.JavaThreadStatistics`, `jdk.ThreadCPULoad` — carga de CPU por thread e número de threads ativas;
- `jdk.FileRead` — eventos de leitura de arquivo (bytes lidos, duração).

2.2.5 Microbenchmark em Erlang

Os microbenchmarks Erlang utilizam `timer:tc/1`, função nativa do OTP que mede o tempo de execução de uma fun com resolução de microssegundos. São executadas 1 iteração de aquecimento e 3 de medição, com o dataset carregado uma única vez em memória antes de qualquer medição para eliminar o custo de I/O dos resultados. São medidos três casos equivalentes ao JMH: execução completa do algoritmo (B1), `closest_centroid` isolado (B3) e `squared_distance` isolado (B4).

2.2.6 Profile em Erlang

O profile Erlang coleta métricas de tempo e GC via funções nativas do OTP, sem necessidade de ferramentas externas. São capturados: tempo de CPU acumulado (`erlang:statistics(runtime)`), tempo de parede (`erlang:statistics(wall_clock)`), número de coletas GC e palavras reclamadas (`erlang:statistics(garbage_collection)`), e utilização média dos schedulers BEAM (`erlang:statistics(scheduler_wall_time)`). O profile cobre uma execução completa do algoritmo com $k = 10$ e `maxIter=10`.

3 Implementação Serial

3.1 Implementação em Java

3.1.1 Descrição

A implementação serial em Java é composta por quatro classes principais, cada uma com responsabilidade bem delimitada:

- **CsvDataset** — lê o cabeçalho do arquivo CSV e expõe os dados linha a linha via `forEachPoint`, sem carregar o dataset inteiro em memória;
- **KMeans** — contém o algoritmo K-Means: inicialização dos centróides, laço de atribuição e recomputo, verificação de convergência e avaliação final;
- **KMeansResult** — registro imutável que armazena os centróides finais, tamanhos de cluster, número de iterações e soma dos erros quadráticos;
- **Main** — ponto de entrada; configura $k = 10$, `maxIter = 10` e `tolerance = 10^{-6}` e imprime o resumo dos clusters ao final.

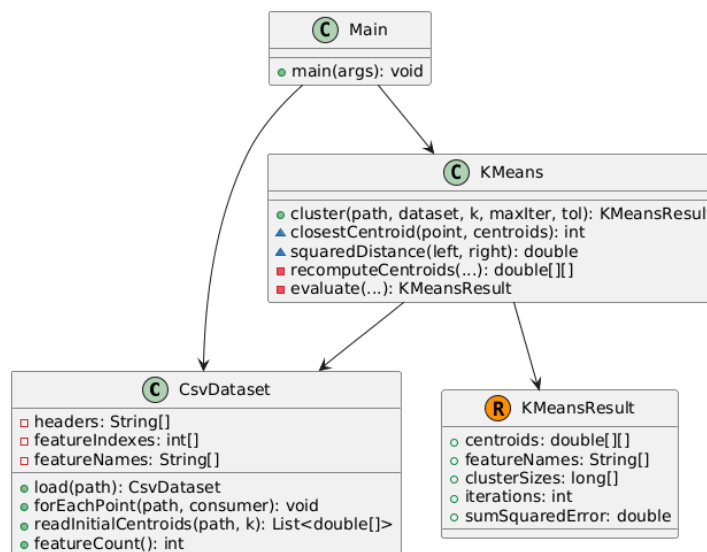


Figure 2: Diagrama de classes da implementação serial em Java.

O dataset é processado em modo *streaming*: a cada iteração do algoritmo, `forEachPoint` abre o arquivo, lê e descarta o cabeçalho, e entrega cada linha decodificada ao consumidor sem acumular pontos na memória heap. A Figura 3 mostra essa interface.

```

public void forEachPoint(Path path, PointConsumer consumer) throws IOException {
    try (BufferedReader reader = Files.newBufferedReader(path)) {
        reader.readLine();

        String line;
        long rowNumber = 1;

        while ((line = reader.readLine()) != null) {
            rowNumber++;

            if (line.isBlank()) {
                continue;
            }

            consumer.accept(parse(line), rowNumber);
        }
    }
}

```

Figure 3: Leitura linha a linha do dataset (`CsvDataset.forEachPoint`).

O laço principal do algoritmo, mostrado na Figura 4, percorre todos os pontos a cada iteração para acumular somas parciais e contagens por cluster, e em seguida recomputa os centróides dividindo as somas pelas contagens.

```

double[][] sums = new double[k][dataset.featureCount()];
long[] counts = new long[k];
double[][] activeCentroids = centroids;

dataset.forEachPoint(datasetPath, (point, rowNumber) -> {
    int closest = closestCentroid(point, activeCentroids);
    counts[closest]++;
    addPoint(sums[closest], point);
});

double[][] updatedCentroids = recomputeCentroids(centroids, sums, counts);

```

Figure 4: Laço de atribuição e recomputo de centróides (`KMeans.cluster`).

A função `closestCentroid` (Figura 5) varre os k centróides para cada ponto e retorna o índice do mais próximo usando distância euclidiana ao quadrado, evitando a raiz quadrada desnecessária. Essa função é o principal ponto quente da CPU, executada $N \times k$ vezes por iteração.

```

static int closestCentroid(double[] point, double[][] centroids) {
    int bestIndex = 0;
    double bestDistance = squaredDistance(point, centroids[0]);

    for (int i = 1; i < centroids.length; i++) {
        double distance = squaredDistance(point, centroids[i]);

        if (distance < bestDistance) {
            bestDistance = distance;
            bestIndex = i;
        }
    }

    return bestIndex;
}

static double squaredDistance(double[] left, double[] right) {
    double total = 0.0d;

    for (int i = 0; i < left.length; i++) {
        double delta = left[i] - right[i];
        total += delta * delta;
    }

    return total;
}

```

Figure 5: Busca do centróide mais próximo e distância euclidiana ao quadrado.

3.1.2 Resultados

Avaliação com Microbenchmark Os microbenchmarks foram executados com JMH 1.37 sobre JDK 25.0.1 (OpenJDK 64-Bit Server VM), com 1 fork, 1 iteração de aquecimento de 10 s e 3 iterações de medição de 10 s cada. O dataset utilizado possui aproximadamente 1 GB de dados de pedidos sintéticos.

ID	Benchmark	Média	Erro ($\pm$)	Unidade
B1	Execução completa ($k=10$, $\text{maxIter}=20$)	212,57	34,48	s/op
B2	Custo por iteração ($k=10$, $\text{maxIter}=5$)	60,05	2,46	s/op
B3	<code>closestCentroid</code> ($k=10$, 38 dims)	96,82	5,59	ns/op
B4	<code>squaredDistance</code> (38 dims)	9,31	1,41	ns/op

Table 6: Resultados JMH para a implementação serial em Java.

B2 mostra que cada iteração leva cerca de 12 s, dominado pela releitura do CSV a cada ciclo. B3 e B4 mostram que o cálculo aritmético é rápido (96 ns e 9 ns por chamada). O gargalo não está na CPU, mas na leitura e no parsing do arquivo.

Avaliação com Macrobenchmark Os macrobenchmarks foram executados com JMeter 5.6.3 sobre JDK 25.0.1, com $k = 10$, $\text{maxIter} = 3$ e $\text{tol} = 0,0$. Cada configuração (GC $\times$ threads) rodou por 120 s com ramp-up de 30 s. A Tabela 7 apresenta a throughput obtida em cada combinação.

Threads	G1GC	ZGC	ParallelGC	Unidade
1	1,56	1,50	1,52	execuções/min
2	2,25	2,01	2,26	execuções/min
4	2,70	2,54	2,79	execuções/min
8	3,44	3,23	3,51	execuções/min
16	4,66	4,40	4,85	execuções/min

Table 7: Throughput por número de threads e coletor de GC — implementação serial em Java.

O throughput cresce com o número de threads JMeter, mas com retornos decrescentes: de 1 para 16 threads, o ganho é de $\approx 3\times$. A latência piora com mais threads, pois todas as instâncias leem o mesmo arquivo de ≈ 1 GB ao mesmo tempo, saturando a I/O do disco.

Avaliação de Profile O profile foi capturado com JFR (`settings=profile`) sobre uma execução completa do algoritmo serial ($k = 10$, `maxIter=3`, G1GC, dataset ≈ 1 GB). A gravação durou 118 s e foi analisada com a ferramenta `jfr` do JDK 25.

Hot Methods. A Tabela 8 apresenta os métodos com maior frequência de amostragem de CPU (`jdk.ExecutionSample`, 11.054 amostras no total).

Método	Amostras	CPU (%)	Categoria
<code>FloatingDecimal.readJavaFormatString</code>	2784	25,2	Parsing CSV
<code>String.split</code>	1873	16,9	Parsing CSV
<code>FloatingDecimal.copyDigits</code>	1817	16,4	Parsing CSV
<code>FloatingDecimal.parseDouble</code>	1782	16,1	Parsing CSV
<code>FloatingDecimal.doubleValue</code>	831	7,5	Parsing CSV
<code>KMeans.squaredDistance</code>	638	5,8	Algoritmo
<code>BufferedReader.readLine</code>	609	5,5	I/O
<code>KMeans.closestCentroid</code>	205	1,9	Algoritmo
<code>KMeans.addPoint</code>	33	0,3	Algoritmo

Table 8: Hot methods na implementação serial em Java (JFR, 11.054 amostras).

O parsing CSV (`String.split` + `Double.parseDouble` e suas dependências internas) consome $\approx 82\%$ do tempo de CPU, revelando que o gargalo real não é a leitura de disco em si, mas a conversão de texto para `double` linha a linha. O código aritmético do K-Means (`squaredDistance` + `closestCentroid` + `addPoint`) representa apenas $\approx 8\%$ — coerente com os resultados JMH de B3 (96 ns/op) e B4 (9 ns/op).

GC. Foram realizadas 1.099 coletas Young GC em 118 s (9,3/s), com pausa total de 805 ms — apenas 0,68% do tempo de execução. O heap oscila entre ≈ 150 MB e ≈ 5 MB após cada coleta, reflexo da alta taxa de criação e descarte de objetos `String` durante o parsing. O GC não é um gargalo.

Threads. Uma única thread de trabalho executa todo o algoritmo; as demais 7 threads são internas da JVM (GC, compilador JIT, etc.). Não há contenção nem paralelismo.

Síntese. O principal gargalo é o parsing CSV (`String.split` + `Double.parseDouble`), que consome $\approx 82\%$ do CPU. O cálculo aritmético do K-Means representa apenas $\approx 8\%$.

3.2 Implementação em Erlang

Uma implementação serial em Erlang não foi realizada. A linguagem é projetada em torno do modelo de atores: processos são a unidade fundamental de execução e toda computação significativa ocorre por troca de mensagens entre eles. Escrever um K-Means verdadeiramente serial em Erlang exigiria contornar deliberadamente os mecanismos centrais da linguagem, produzindo código artificial e não representativo. A implementação Erlang é apresentada diretamente na Seção 4 como versão concorrente.

3.3 Discussão

Trechos caros e candidatos à concorrência. O JFR mostra que $\approx 82\%$ do CPU é gasto com parsing CSV (`String.split` + `Double.parseDouble`). O laço de atribuição, que chama `closestCentroid` $N \times k$ vezes por iteração, é independente entre pontos e é o principal candidato à paralelização. A releitura do arquivo a cada iteração (`forEachPoint`) é outro gargalo que poderia ser eliminado carregando o dataset em memória.

Comparação de coletores de GC. O ParallelGC tem o maior throughput em todos os níveis de carga, seguido pelo G1GC. O ZGC fica levemente atrás em todos os casos pelas barreiras de escrita do seu algoritmo. No geral, o impacto do GC é pequeno — as pausas somam menos de 1% do tempo total.

4 Implementação Concorrente

4.1 Implementação em Java

4.1.1 Descrição

A estrutura geral do algoritmo é mantida em relação à versão serial: inicializar centróides, iterar entre atribuição e recomputo, verificar convergência. O que muda é *como* os pontos são processados dentro de cada iteração.

Na versão serial, a thread principal lê cada ponto diretamente do arquivo, chama `closestCentroid` e acumula os resultados em arrays compartilhados. Na versão concorrente, a leitura ainda é feita sequencialmente, mas os pontos são agrupados em lotes (`BATCH_SIZE = 5.000`) e cada lote é submetido ao pool de workers como uma tarefa independente. Os workers acumulam em arrays locais e retornam um `PartialResult`; a thread coordenadora reduz todos os parciais após coletar os `Futures`.

Esse padrão — dispatch em lotes, acumulação local, redução sequencial — elimina a necessidade de sincronização nas regiões críticas CR1–CR4 sem uso de `synchronized` ou variáveis atômicas. A chamada `Future.get()` garante a relação happens-before entre a escrita dos workers e a leitura da coordenadora.

Uma cópia imutável dos centróides (**snapshot**) é tirada antes do dispatch de cada iteração, garantindo que todos os workers vejam o mesmo estado de centróides durante a iteração em andamento.

Foram implementadas três variantes concorrentes:

- **v2-platform** — pool fixo de N platform threads para os workers e a thread `main` como coordenadora.
- **v2-virtual** — pool fixo de N virtual threads para os workers, mesma estrutura de v2-platform.
- **v2-hybrid** — pool fixo de N platform threads como workers CPU-bound e uma virtual thread como coordenadora (`newVirtualThreadPerTaskExecutor`). A virtual thread desmonta automaticamente durante `readLine` (I/O de disco) e durante cada `Future.get()` (park), liberando o carrier sem bloquear o `ForkJoinPool`.

As classes `CsvDataset`, `KMeansResult` e `Main` não foram alteradas. As adições ficaram concentradas em `KMeans.java`: dois records privados (`PartialResult`, `EvaluatePartial`) e três métodos novos (`runClustering`, `processAssignBatch`, `processEvaluateBatch`). O método `evaluate` foi refatorado para também usar o pool de workers. A classe `KMeansSampler` foi adicionada como adaptador JMeter para os macrobenchmarks.

O diagrama de classes da Figura 6 mostra as classes do projeto com as adições da versão concorrente em destaque.

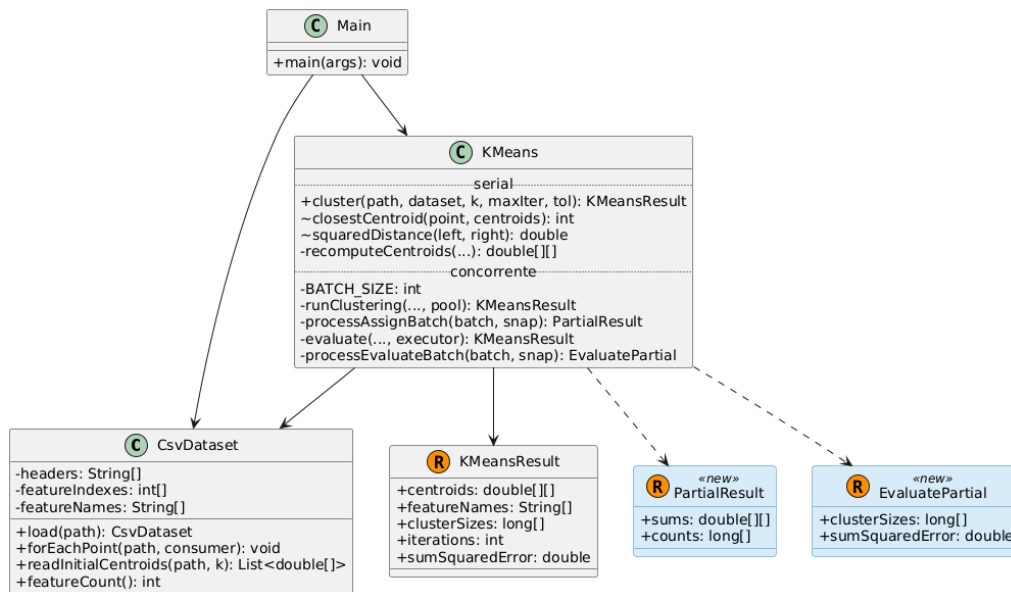


Figure 6: Diagrama de classes da implementação concorrente em Java. Elementos destacados são novos em relação à versão serial.

A Figura 7 mostra a criação dos dois executores e a submissão do método coordenador à virtual thread.

```

int n = Runtime.getRuntime().availableProcessors();
ExecutorService computePool = Executors.newFixedThreadPool(n, Thread.ofPlatform().factory());
ExecutorService coordinator = Executors.newFixedThreadPool(n, Thread.ofVirtual().factory());

try {
    return coordinator.submit(
        () -> runClustering(datasetPath, dataset, k, maxIterations, tolerance,
            centroids, computePool)
    ).get();
} finally {
    coordinator.shutdown();
    computePool.shutdown();
}

```

Figure 7: Criação dos executores e submissão do coordenador (`KMeans.cluster`).

A Figura 8 mostra o laço de dispatch de lotes dentro de `runClustering`: a leitura sequencial acumula pontos até atingir `BATCH_SIZE`, quando cada lote é submetido ao pool e o buffer é esvaziado.

```

double[][] snapshot = copy(centroids);
List<Future<PartialResult>> futures = new ArrayList<>();
List<double[]> batch = new ArrayList<>(BATCH_SIZE);

dataset.forEachPoint(datasetPath, (point, rowNum) -> {
    if (batch.size() == BATCH_SIZE) {
        List<double[]> current = List.copyOf(batch);
        futures.add(computePool.submit(() -> processAssignBatch(current,
snapshot)));
        batch.clear();
    }
});

// reduce
for (Future<PartialResult> f : futures) {
    PartialResult pr = f.get();
    for (int c = 0; c < k; c++) {
        counts[c] += pr.counts()[c];
        for (int d = 0; d < sums[c].length; d++)
            sums[c][d] += pr.sums()[c][d];
    }
}

```

Figure 8: Dispatch de lotes e coleta de futures (KMeans.runClustering).

A Figura 9 mostra `processAssignBatch`, o método executado por cada worker: usa arrays locais (`localSums`, `localCounts`) sem compartilhamento, retornando um `PartialResult` imutável.

```

private static PartialResult processAssignBatch(List<double[]> batch,
double[][] centroids) {
    int k = centroids.length, d = centroids[0].length;
    double[][] localSums = new double[k][d];
    long[] localCounts = new long[k];

    for (double[] point : batch) {
        int c = closestCentroid(point, centroids);
        localCounts[c]++;
        addPoint(localSums[c], point);
    }
    return new PartialResult(localSums, localCounts);
}

```

Figure 9: Processamento local de um lote sem arrays compartilhados (KMeans.processAssignBatch).

4.1.2 Resultados

Avaliação de Condição de Corrida Foram identificadas quatro regiões críticas na implementação concorrente, todas relacionadas a acumulações que seriam problemáticas caso múltiplas threads compartilhassem os mesmos arrays de escrita:

ID	Local	Operação	Risco
CR1	processAssignBatch	counts[c]++	lost update em long[]
CR2	processAssignBatch	sums[c][d] += v	lost update em double[] []
CR3	processEvaluateBatch	sse += v	acumulação não atômica de double
CR4	processEvaluateBatch	clusterSizes[c]++	lost update em long[]

Table 9: Regiões críticas identificadas na implementação concorrente.

A Figura 10 mostra `processAssignBatch`, o método executado por cada worker. As operações `localCounts[c]++` (CR1) e `addPoint(localSums[c], point)` — que internamente faz `sums[c][d] += v` (CR2) — seriam race conditions se os arrays fossem compartilhados entre workers. A implementação usa arrays locais por tarefa, tornando-as thread-safe.

```
private static PartialResult processAssignBatch(List<double[]> batch, double[][] centroids) {
    int k = centroids.length, d = centroids[0].length;
    double[][] localSums = new double[k][d]; // local - sem compartilhamento
    long[] localCounts = new long[k]; // local - sem compartilhamento

    for (double[] point : batch) {
        int c = closestCentroid(point, centroids);
        localCounts[c]++; // CR1 - race se fosse array compartilhado
        addPoint(localSums[c], point); // CR2 - race se fosse array compartilhado
    }

    return new PartialResult(localSums, localCounts);
}
```

Figure 10: Operações CR1 e CR2 em `processAssignBatch`: `localCounts[c]++` e `addPoint` seriam race conditions se os arrays fossem compartilhados.

Os testes foram executados com JCTestress 0.16, preset `default`, 6 forks por teste e 5 iterações de 1.000 ms por fork. A Tabela 10 apresenta o resultado consolidado.

ID	Resultado	Outcome FORBIDDEN	Freq. máx.
JCS1	PASSOU	nenhum	—
JCS2	PASSOU	nenhum	—
JCS3	FALHOU (esperado)	"1" lost update em counts[0]++	28,31%
JCS4	FALHOU (esperado)	"1.0" lost update em sums[0]+=1.0	27,10%

Table 10: Resultados JCTestress — regiões críticas da implementação concorrente.

JCS1 e JCS2 não produziram nenhum outcome FORBIDDEN: a leitura concorrente é segura. JCS3 e JCS4 falharam como esperado: lost updates ocorrem em até 28% das amostras com compilação C2 e em 0,01% com o Interpreter.

Avaliação com Microbenchmark Os microbenchmarks foram executados com JMH 1.37 sobre JDK 25.0.1, com 1 fork, 1 iteração de aquecimento de 10 s e 3 iterações de medição de 10 s cada. A Tabela 11 compara as três variantes com a versão serial.

ID (Un.)	Serial	v2-platform	v2-virtual	v2-hybrid
B1 (s)	212,57	214,87 ($\pm 29,10$)	211,64 ($\pm 260,68$)	215,69 ($\pm 59,78$)
B2 (s)	60,05	60,69 ($\pm 6,41$)	71,61 ($\pm 227,05$)	60,44 ($\pm 5,33$)
B3 (ns)	96,82	102,24	101,11	102,07
B4 (ns)	9,31	9,70	9,58	9,87

Table 11: Resultados JMH — serial vs. variantes v2 (média; B1/B2 com erro $\pm$).

Nenhuma variante reduz o tempo em relação ao serial: o parsing CSV sequencial domina B1 e B2 em todas elas. A v2-virtual tem variância muito alta (± 260 s em B1) porque o `ForkJoinPool` não é adequado para tarefas CPU-bound que nunca bloqueiam. B3 e B4 são praticamente inalterados nas três variantes.

Avaliação com Macrobenchmark Os macrobenchmarks foram executados com JMeter 5.6.3 sobre JDK 25.0.1 ($k = 10$, `maxIter=3`, `tol=0,0`), 120 s por configuração e ramp-up de 30 s. A Tabela 12 mostra o throughput com G1GC para as três variantes.

Threads	v2-platform	v2-virtual	v2-hybrid
1	1,65	1,59	1,60
2	1,95	2,17	2,13
4	3,92	3,29	2,67
8	3,08	4,00	3,44
16	4,49	4,55	4,53

Table 12: Throughput (execuções/min, G1GC) por variante v2 e número de threads JMeter.

Com 1 thread JMeter o throughput é $\approx 1,6$ exec/min para todas as variantes. Com 4 threads, v2-platform lidera (3,92); com 8, v2-virtual supera as demais (4,00) porque os carriers do `ForkJoinPool` evitam a saturação do escalonador do kernel que ocorre com $8 \times N$ platform threads reais. Em 16 threads todas convergem para $\approx 4,5$ exec/min. O ZGC fica levemente abaixo de G1GC e ParallelGC em todos os casos pelas barreiras de escrita do seu algoritmo.

Avaliação de Profile O profile foi capturado com JFR (`settings=profile`) sobre uma execução completa do algoritmo ($k = 10$, `maxIter=3`, G1GC, dataset ≈ 1 GB). A gravação durou ≈ 113 s e foi analisada com a ferramenta `jfr` do JDK 25.

v2-platform e v2-virtual. Em ambas as variantes, a thread `main` domina o perfil de CPU: 86,6% em v2-platform (parsing CSV) e 99,8% em v2-virtual. Na v2-virtual os workers aparecem em apenas 21 das 10.164 amostras, pois cada lote termina em menos de 10 ms — abaixo do intervalo de amostragem do JFR — deixando as virtual threads invisíveis ao profiler. GC: ≈ 1.000 coletas Young em ≈ 110 s ($< 1\%$ de pausa). Não é gargalo.

Método	CPU (%)	Categoria
KMeans.squaredDistance	83,4 (workers)	Algoritmo
KMeans.closestCentroid	14,2 (workers)	Algoritmo
KMeans.addPoint	1,5 (workers)	Algoritmo
coordenador virtual	0,1	Coordenação

Table 13: Hot methods na implementação v2-hybrid (JFR, 2.133 amostras).

v2-hybrid. O coordenador virtual desmonta durante `readLine` e `Future.get()`, tornando o parsing CSV invisível ao JFR. Os 16 workers de plataforma concentram 99,9% das amostras, com `squaredDistance` e `closestCentroid` no topo. GC: 1.072 coletas em 140 s (0,84% de pausa). Não é gargalo.

4.2 Discussão

Condições de corrida. Os testes JCSstress identificaram quatro regiões críticas (CR1–CR4), todas associadas a acumulações em arrays compartilhados. JCS3 e JCS4 confirmaram lost updates em até 28% das amostras quando a sincronização é omitida. A solução adotada — acumulação local por worker com redução após `Future.get()` — elimina a race condition sem uso de `synchronized` ou `atomics`.

Comparação de abordagens — microbenchmark. A Tabela 14 compara as três variantes com a versão serial.

Abordagem	B1 (s/op)	B2 (s/op)	Erro B1	Erro B2
Serial	212,57	60,05	±34,48	±2,46
v2-platform	214,87	60,69	±29,10	±6,41
v2-virtual	211,64	71,61	±260,68	±227,05
v2-hybrid	215,69	60,44	±59,78	±5,33

Table 14: Comparativo JMH entre abordagens (s/op).

Nenhuma abordagem reduz o tempo médio em relação ao serial: o parsing CSV sequencial é o gargalo comum a todas. v2-virtual apresenta variância muito alta (± 260 s em B1), reflexo do `ForkJoinPool` ao lidar com tarefas CPU-bound. v2-platform e v2-hybrid têm variância próxima ao serial.

Comparação de abordagens — macrobenchmark. A Tabela 15 compara o throughput (G1GC) por número de threads JMeter.

Abordagem	1 t	2 t	4 t	8 t	16 t
v2-platform	1,65	1,95	3,92	3,08	4,49
v2-virtual	1,59	2,17	3,29	4,00	4,55
v2-hybrid	1,60	2,13	2,67	3,44	4,53

Table 15: Throughput (execuções/min, G1GC) por abordagem e número de threads JMeter.

Com poucos threads JMeter (1–2), as três variantes são equivalentes. Com 4 threads, v2-platform tem o maior throughput (3,92 exec/min). Com 8 threads, v2-virtual supera as demais (4,00 exec/min): seus workers são virtual threads multiplexadas sobre N carriers,

evitando a saturação do escalonador de kernel que ocorre com $8 \times N$ platform threads reais. Em 16 threads, todas convergem para $\approx 4,5$ exec/min.

Comparação de coletores de GC. O GC tem impacto pequeno em todas as variantes (pausas $< 1\%$ do tempo total). O ZGC fica levemente abaixo de G1GC e ParallelGC em configurações com muitas threads simultâneas, pelo custo das barreiras de escrita do seu algoritmo. G1GC e ParallelGC têm desempenho semelhante entre si.

5 Implementação Concorrente — Dados em Memória

5.1 Implementação em Java

5.1.1 Descrição

A v3 parte de v2-virtual, a variante com melhor desempenho na seção anterior, e elimina o principal gargalo identificado no profile: o parsing CSV repetido a cada iteração. A mudança central é carregar todos os pontos do dataset em memória uma única vez antes do loop de iterações.

`CsvDataset.loadAllPoints()` lê o arquivo e retorna um `double[][]` com todos os pontos. O método `KMeans.cluster()` ganhou uma segunda assinatura que recebe esse array diretamente, sem tocar em disco durante o clustering. `KMeansBenchmark` pré-carrega o array no `@Setup(Level.Trial)`, excluindo a leitura do CSV do tempo medido. `KMeansSampler` usa um campo `static volatile double[][] sharedPoints` com double-checked locking para que todos os threads JMeter compartilhem uma única cópia dos dados em memória.

Em vez de criar lotes de `BATCH_SIZE` pontos e coletar uma lista de `Futures`, a v3 divide o array em N chunks de tamanho fixo (onde N é o número de processadores lógicos) e usa `CountDownLatch(N)` para coordenar a conclusão. Cada worker recebe índices `from/to` e opera diretamente sobre o array compartilhado (somente leitura) com acumulação em arrays locais.

Componente	Alteração
<code>CsvDataset</code>	adicionado <code>loadAllPoints()</code>
<code>KMeans</code>	nova assinatura <code>cluster(double[][], ...)</code> ; <code>CountDownLatch</code> em vez de <code>Futures</code> ; <code>processChunk</code> em vez de <code>processAssignBatch</code> ; <code>DoubleAdder</code> para SSE
<code>KMeansBenchmark</code>	<code>points</code> pré-carregado em <code>@Setup(Level.Trial)</code>
<code>KMeansSampler</code>	<code>static volatile sharedPoints</code> com double-checked locking

Table 16: Alterações da v3 em relação à v2-virtual.

5.1.2 Resultados

Avaliação de Condição de Corrida Os mesmos quatro testes JCSstress (JCS1–JCS4) da seção anterior foram executados sobre a v3. A estrutura de acumulação local por worker foi mantida: cada chunk opera em `localSums` e `localCounts` privados, sem compartilhamento.

ID	Resultado	Outcome FORBIDDEN	Freq. máx.
JCS1	PASSOU	nenhum	—
JCS2	PASSOU	nenhum	—
JCS3	FALHOU (esperado)	"1" lost update em <code>counts[0]++</code>	4,05%
JCS4	FALHOU (esperado)	"1.0" lost update em <code>sums[0]+=1.0</code>	5,95%

Table 17: Resultados JCSstress — regiões críticas da v3.

JCS1 e JCS2 passam: leitura concorrente do array de centroides é segura. JCS3 e JCS4 falham como esperado, confirmando que a race condition existe quando os arrays

são compartilhados — o que a v3 evita da mesma forma que a v2.

Avaliação com Microbenchmark Os microbenchmarks foram executados com JMH 1.37 sobre JDK 25.0.1 (OpenJDK 64-Bit Server VM), com 1 fork, 1 iteração de aquecimento de 10 s e 3 iterações de medição de 10 s cada. O dataset foi pré-carregado em memória antes do início das medições (`@Setup(Level.Trial)`).

ID	Benchmark	Serial	v3	Erro ($\pm$)	Un.
B1	Execução completa ($k=10$, iter=20)	212,57	4,55	17,98	s/op
B2	Por iteração ($k=10$, iter=5)	60,05	1,60	0,17	s/op
B3	closestCentroid (38 dims)	96,82	96,82	—	ns/op
B4	squaredDistance (38 dims)	9,31	9,31	—	ns/op

Table 18: Resultados JMH — serial vs. v3 (dados em memória).

B2 cai de 60 s/op para 1,60 s/op: a eliminação do parsing CSV por iteração reduz o custo em $\approx 37\times$. B1 tem variância elevada (± 18 s) por depender do número de iterações até convergência. B3 e B4 são inalterados (computação pura).

Avaliação com Macrobenchmark Os macrobenchmarks foram executados com JMeter 5.6.3 sobre JDK 25.0.1, com $k = 10$, $\text{maxIter} = 3$ e $\text{tol} = 0,0$. O dataset é carregado em memória uma única vez por processo JMeter. Cada configuração rodou por 120 s com ramp-up de 30 s, utilizando ParallelGC.

Threads JMeter	Execuções	Throughput (exec/min)	Latência média (ms)
1	105	52,5	1.145
2	112	56,0	2.010
4	116	57,1	3.796
8	119	57,2	7.457
16	134	62,3	13.374

Table 19: Throughput e latência por número de threads JMeter — v3, ParallelGC.

Com 1 thread JMeter, o throughput é 52,5 exec/min. O ganho com mais threads é moderado ($52,5 \rightarrow 62,3$ exec/min de 1 para 16 threads) porque cada execução já satura todos os núcleos internamente. Nenhum erro foi registrado em nenhuma configuração.

Avaliação de Profile O profile foi capturado com JFR (`settings=profile`) sobre uma execução completa do algoritmo ($k = 10$, $\text{maxIter}=3$, dataset $\approx 1,75$ GB em memória). A gravação foi analisada com a ferramenta `jfr` do JDK 25.

I/O. Nenhum evento `jdk.FileRead` foi registrado durante o clustering. O arquivo CSV é lido apenas na fase de inicialização; as iterações operam exclusivamente sobre o array em memória.

GC. Foram realizadas 60 coletas (51 Young G1New + 9 Old G1Old) com pausa total de 1.024 ms e pausa média de 17 ms por evento. O heap atingiu pico de 2,7 GB, correspondente ao dataset carregado ($\approx 1,75$ GB) mais overhead de trabalho.

Threads. O pool de virtual threads usa os N carriers do ForkJoinPool interno. Chunks terminam em ≈ 100 –200 ms, sendo facilmente visíveis ao JFR ao contrário dos lotes de 5.000 pontos da v2.

Síntese. O parsing CSV desaparece completamente do perfil de execução. O tempo é gasto inteiramente em computação aritmética (`closestCentroid`, `squaredDistance`) distribuída entre os N workers.

5.2 Implementação em Erlang

5.2.1 Descrição

A implementação concorrente em Erlang segue o modelo de atores da BEAM: processos isolados que se comunicam exclusivamente por troca de mensagens, sem memória compartilhada.

O dataset é carregado uma única vez na memória pelo processo principal, com cada ponto representado como um binário compacto de 304 bytes (`«V1:64/float, V2:64/float, ..., V38:64/float»`). Binários maiores que 64 bytes são armazenados em um heap compartilhado com contagem de referências, não sendo copiados ao serem enviados entre processos — equivalente ao `double[][]` do Java em termos de custo de distribuição.

São criados $N = 16$ processos workers de longa duração (*long-lived*), um por scheduler BEAM disponível. Cada worker recebe seu chunk de pontos uma única vez na inicialização e o mantém em seu heap local. Por iteração:

1. O processo principal envia `{iterate, Centroids, From}` a cada worker.
2. Cada worker percorre seu chunk, calcula o centróide mais próximo com `closest_centroid` e acumula somas e contagens parciais locais.
3. Cada worker envia `{partial, Sums, Counts}` de volta ao principal.
4. O principal coleta N respostas, mescla os parciais e recomputa os centróides.

Os centróides são mantidos como listas de floats (`[float()]`), pois são pequenos ($k \times d \times 8$ bytes ≈ 3 KB) e precisam suportar atualização por divisão elemento a elemento.

5.2.2 Resultados

Condição de Corrida Por construção, o modelo de atores Erlang não possui memória compartilhada entre processos. Cada worker acumula somas e contagens em seu próprio heap, e o processo principal é o único a ler e escrever os centróides. Não há race conditions possíveis, dispensando sincronização explícita.

Avaliação com Microbenchmark Os microbenchmarks foram executados com `timer:tc` sobre OTP 25 (BEAM JIT habilitado), com 1 execução de aquecimento e 3 execuções de medição, todas com $k = 10$ e `maxIter` = 5. O dataset (≈ 1 GB) foi carregado em memória uma única vez antes das medições.

ID	Benchmark	Erlang concorrente	Unidade
B1	Execução completa ($k=10$, $\text{maxIter}=5$)	106.344,7	ms/op
B3	<code>closest_centroid</code> ($k=10$, 38 dims)	15.976,0	ns/op
B4	<code>squared_distance</code> (38 dims)	1.052,7	ns/op

Table 20: Resultados de microbenchmark para a implementação concorrente em Erlang (16 workers, `timer:tc`).

B1 reflete o custo total de 5 iterações sobre 5,75 M pontos com 16 workers. B3 e B4 isolam a aritmética por ponto: `closest_centroid` leva 15,98 μ s e `squared_distance` 1,05 μ s por chamada.

Avaliação de Profile O profile foi capturado com `erlang:statistics` sobre uma execução completa ($k = 10$, $\text{maxIter}=10$, 16 workers, dataset ≈ 1 GB). A execução durou 188,7s de parede, equivalente a $\approx 18,9$ s por iteração.

Schedulers. A utilização média dos schedulers BEAM foi de 45,7%. Os workers ficam ociosos enquanto o processo principal executa `collect_partial` e recomputa centróides serialmente; durante esse intervalo os 16 workers aguardam na caixa de mensagens sem consumir CPU.

GC. Foram realizadas 2.129.920 coletas GC em 188,7s (11.329/s), com 246,5 bilhões de palavras reclamadas ($\approx 1,97$ TB). O volume elevado reflete a alta taxa de alocação por ponto: `update_nth` reconstrói parcialmente a lista de somas a cada ponto processado, gerando lixo proporcional a $k \times d$ por chamada. O GC per-processo da BEAM coleta esse lixo de forma isolada, sem pausas globais, mas com custo acumulado significativo.

Síntese. O gargalo é duplo: aritmética por ponto (`squared_distance/closest_centroid`, 113–165 $\times$ mais lenta que Java) e redução serial (main process coleta parciais sequencialmente, mantendo schedulers ociosos $\approx 54\%$ do tempo).

5.3 Discussão

Java v2 $\rightarrow$ v3: eliminação do gargalo de I/O. Nas versões v2, o parsing CSV consumia 86–99% do CPU em cada iteração. Na v3, nenhum evento de leitura de arquivo é registrado durante o clustering. A Tabela 21 resume o impacto.

Abordagem	B1 (s/op)	B2 (s/op)	Erro B1	Erro B2
Serial	212,57	60,05	$\pm 34,48$	$\pm 2,46$
v2-virtual	211,64	71,61	$\pm 260,68$	$\pm 227,05$
v3	4,55	1,60	$\pm 17,98$	$\pm 0,17$

Table 21: Comparativo JMH: serial, v2-virtual e v3 (s/op).

B2 cai de 60 s para 1,6 s (37 $\times$): o custo dominante deixa de ser I/O e passa a ser computação aritmética pura. A variância de B1 colapsa de ± 260 s (v2-virtual) para ± 18 s (v3): sem I/O, cada iteração tem carga uniforme e previsível. No macrobenchmark, o throughput com 1 thread JMeter salta de 1,62 para 52,5 exec/min (32 $\times$); com 16 threads o ganho cai para 14 $\times$ porque os processos concorrentes competem pelos mesmos núcleos. Como trade-off, o heap pico cresce de ≈ 250 MB (v2) para $\approx 2,7$ GB (v3), porém o número de coletas GC cai de 1.000+ para 60, pois há muito menos alocação transiente por iteração.

Java v3 vs. Erlang: modelo de memória vs. modelo de atores. Com o I/O eliminado em ambas as implementações, a comparação entre Java v3 e Erlang torna-se justa. A Tabela 22 expõe a diferença.

ID	Java v3	Erlang	Fator ($\times$ mais lento)	Un.
B1/B2 (maxIter=5)	1.600,0	106.344,7	66,5	ms/op
B3	96,82	15.976,0	165	ns/op
B4	9,31	1.052,7	113	ns/op

Table 22: Comparativo Java v3 vs. Erlang concorrente (ambos em memória, 16 workers).

A diferença de $66,5\times$ no tempo total é explicada inteiramente pela aritmética por ponto (B3/B4): o JIT da JVM compila loops sobre `double[]` em código nativo com potencial de vetorização SIMD, enquanto a BEAM desempacota floats de binários elemento a elemento (`<V:64/float, Rest/binary>`) sem esse caminho de otimização.

Além da aritmética, dois fatores estruturais amplificam a diferença. Primeiro, a utilização dos schedulers BEAM ficou em apenas 45,7%: enquanto o processo principal executa `collect_partial` e recomputa centróides serialmente, os 16 workers aguardam na caixa de mensagens sem consumir CPU — uma ineficiência inerente ao padrão de coordenação centralizada. Em Java v3, os workers operam sobre um array compartilhado (somente leitura) e o coordenador usa `CountDownLatch`, mantendo todos os núcleos ativos até o final de cada iteração. Segundo, o GC Erlang realizou 2,13 milhões de coletas (por processo, sem pausas globais) contra apenas 60 coletas na JVM: `update_nth` reconstrói parcialmente a lista de somas a cada ponto, gerando lixo proporcional a $k \times d$ por chamada — custo inexistente em Java, que acumula em arrays mutáveis locais.

Quando cada modelo vence. O K-Means é uma carga de trabalho CPU-bound com comunicação mínima entre iterações — o caso em que memória compartilhada tem vantagem estrutural sobre troca de mensagens. O modelo de atores da Erlang traz isolamento e tolerância a falhas sem sincronização explícita, o que é valioso em sistemas distribuídos e de alta disponibilidade, mas não neste workload numérico intensivo.

6 Discussão Final

6.1 Comparativo Geral das Abordagens

O trabalho evoluiu em três estágios distintos, cada um respondendo a uma limitação do estágio anterior.

Serial. A implementação sequencial estabeleceu a baseline: uma thread lê o CSV, calcula a distância de cada ponto aos centróides e acumula os resultados. O perfil de execução revelou que $\approx 87\%$ do tempo era gasto em parsing CSV (`FloatingDecimal.parseDouble`, `String.split`), e apenas $\approx 13\%$ em computação aritmética. Isso significa que o algoritmo em si é rápido; o gargalo é a entrada de dados.

Concorrente v2 (platform, virtual, hybrid). A paralelização da fase de atribuição (lotes de 5.000 pontos, pool de N workers) não trouxe ganho de tempo por iteração: o parsing CSV seguia sequencial na thread principal, dominando o tempo total. Os microbenchmarks mostraram B2 ≈ 60 s/op para todas as variantes, idêntico ao serial. O macrobenchmark revelou throughput de 1,6–4,6 exec/min dependendo do número de threads JMeter e da variante de GC.

A exploração das três variantes foi útil para entender o comportamento de cada modelo de thread. Platform threads escalonam pelo kernel; virtual threads multiplexam sobre carriers do `ForkJoinPool`. A variante hybrid foi a que mais claramente separou os papéis: o coordenador virtual desmontava durante I/O e `Future.get()`, tornando o parsing CSV invisível ao profiler de CPU — revelando que o problema não era a coordenação, mas a leitura em si.

Concorrente v3 (dados em memória). Carregar o dataset inteiro em `double[][]` antes do loop de iterações eliminou o parsing CSV das medições. B2 caiu de ≈ 60 s para $\approx 1,6$ s ($\approx 37\times$); o throughput saltou de $\approx 1,6$ para ≈ 52 exec/min com 1 thread JMeter. O perfil não registrou nenhum evento de leitura de arquivo durante o clustering. O custo foi o aumento do heap de ≈ 250 MB para $\approx 2,7$ GB.

Erlang concorrente (modelo de atores, dados em memória). Distribui chunks entre 16 workers BEAM, sem memória compartilhada. O custo por iteração ficou em $\approx 18,9$ s ($12\times$ acima do Java v3). O gargalo é a aritmética por ponto: B3 e B4 são $113\text{--}165\times$ mais lentos que em Java, pois a BEAM desempacota floats de binários sem vetorização SIMD. Os schedulers operaram a 45,7% de utilização — a redução centralizada serializa parte de cada iteração.

A Tabela 23 resume os principais números de cada abordagem.

Abordagem	B2 (s/op)	Throughput 1t	Throughput 16t	Heap pico
Serial	60,05	—	—	≈ 250 MB
v2-platform	60,69	1,65/min	4,49/min	≈ 250 MB
v2-virtual	71,61	1,62/min	4,55/min	≈ 250 MB
v2-hybrid	60,44	1,60/min	4,53/min	≈ 250 MB
v3	1,60	52,5/min	62,3/min	$\approx 2,7$ GB
Erlang (16 workers)	18,87	N/A	N/A	$\approx 1,75$ GB

Table 23: Comparativo geral entre abordagens (B2 = custo por iteração; throughput = exec/min, JMeter ParallelGC; Erlang sem macrobenchmark HTTP).

6.2 Pontos Positivos e Negativos de Cada Abordagem

Serial

Positivos. Simples de implementar e depurar. Ausência de condições de corrida por definição. Baixo consumo de memória. Serve como baseline clara para medir ganhos das versões concorrentes.

Negativos. Subutiliza processadores multi-core. A thread principal acumula os dois papéis (I/O e computação) sem possibilidade de sobreposição.

v2 — Concorrência com Lotes e Futures

Positivos. Paraleliza a fase computacionalmente intensiva (atribuição de pontos a centróides) sem exigir sincronização explícita, graças à acumulação local por worker. A estrutura de `Future.get()` fornece garantia de happens-before clara. A variante hybrid demonstrou que virtual threads são adequadas para coordenadores que alternam entre I/O e espera.

Negativos. O parsing CSV sequencial na thread principal continua sendo o gargalo, tornando o ganho da paralelização invisível no tempo de parede. A criação de objetos `List.copyOf` e `PartialResult` por lote gera pressão de GC. A virtual thread em tarefas CPU-bound de curta duração introduz variância elevada no JMH (até ± 260 s), pois o `ForkJoinPool` não foi projetado para tarefas que nunca bloqueiam.

Erlang — Concorrência por Atores

Positivos. Race conditions são impossíveis por construção: processos não compartilham memória, eliminando a necessidade de locks, atomics ou `volatile`. O GC é por processo e sem pausas globais: nenhum stop-the-world afeta todos os workers simultaneamente. Workers de longa duração mantêm os chunks em seus heaps locais sem overhead de cópia, graças à semântica de referência dos binários grandes da BEAM. O modelo de atores escala naturalmente para clusters distribuídos sem mudança de paradigma.

Negativos. A aritmética por ponto é $113\text{--}165\times$ mais lenta que em Java: a BEAM desempacota floats de binários elemento a elemento sem geração de código SIMD. Os schedulers operaram a 45,7% de utilização: a redução centralizada no processo principal (`collect_partial` + `recompute`) é um gargalo serial que mantém os workers ociosos $\approx 54\%$ do tempo. A reconstrução de listas de somas via `update_nth` gerou 2,1 milhões de coletas GC por alocação excessiva de células de lista — custo inexistente em Java, que acumula em arrays mutáveis locais. A ausência de servidor HTTP inviabilizou a comparação de throughput com JMeter nas mesmas condições das versões Java.

v3 — Dados em Memória com `CountDownLatch`

Positivos. Elimina o I/O das iterações, revelando o verdadeiro custo computacional do algoritmo. A divisão em N chunks fixos é mais simples que a criação dinâmica de lotes: sem `List.copyOf`, sem `Future` por lote. `CountDownLatch` é mais direto que coletar e iterar sobre uma lista de `Futures`. O `KMeansSampler` com `sharedPoints` estático permite que múltiplos threads JMeter compartilhem os dados sem alocar múltiplos GBs por thread.

Negativos. O dataset inteiro precisa caber na heap ($\approx 1,75$ GB para $\approx 5,75$ milhões de pontos com 38 features). Isso inviabiliza a abordagem para datasets maiores sem parti-

cionamento externo. O tempo de inicialização (leitura única do CSV) não é contabilizado nas métricas, o que pode ser enganoso se o processo for reiniciado frequentemente.

6.3 Lições Aprendidas

Medir antes de paralelizar. O JFR mostrou que 87% do tempo serial era parsing CSV. Paralelizar apenas o loop aritmético (13% do total) dificilmente produziria ganho perceptível sem antes eliminar o gargalo de I/O.

Gargalos migram. Com o I/O eliminado na v3, o gargalo passou a ser `squaredDistance` e `closestCentroid`. Em iterações futuras, esse é o ponto a otimizar (SIMD, vetorização, redução de dimensionalidade).

Virtual threads não são bala de prata para CPU-bound. Virtual threads brilham com bloqueio I/O, liberando o carrier para reuso. Em tarefas puramente aritméticas, não oferecem vantagem sobre platform threads e podem introduzir variância de escalonamento indesejada.

Acumulação local elimina sincronização. O padrão map-reduce local — cada worker acumula em arrays próprios, a thread coordenadora reduz no final — evitou locks, atomics e `volatile`. O JCSstress confirmou a ausência de race conditions nas operações reais e sua presença ao remover a proteção (JCS3, JCS4).

O modelo de atores não é universal. K-Means é CPU-bound com comunicação mínima entre iterações — perfil que favorece memória compartilhada. O modelo de atores Erlang brilha em sistemas distribuídos, I/O intensivo e alta disponibilidade, onde o isolamento de processos é vantagem estrutural, não custo.

7 Repositórios

Os repositórios do projeto estão disponíveis no GitHub. Cada um corresponde a um componente do trabalho e possui tags marcando as versões entregues.

Implementação Java

Contém o algoritmo K-Means serial e as variantes concorrentes (v2-platform, v2-virtual, v2-hybrid, v3), além dos benchmarks JMH, testes JCStress, macrobenchmark JMeter e profile JFR.

- **URL:** <https://github.com/rarycoringa/concurrent-kmeans-java-application>
- **Tags:** v1 (serial), v2-platform, v2-virtual, v2-hybrid, v3 (dados em memória)

Implementação Erlang

Contém a implementação concorrente em Erlang com o modelo de atores da BEAM, benchmarks com `timer:tc` e profile com `erlang:statistics`.

- **URL:** <https://github.com/rarycoringa/concurrent-kmeans-erlang-application>
- **Tags:** v1 (concorrente, dados em memória)

Gerador de Dataset

Scripts Python para geração do dataset sintético de pedidos utilizado em todos os experimentos ($\approx 5,75$ milhões de linhas, 38 features).

- **URL:** <https://github.com/rarycoringa/concurrent-kmeans-dataset-generator-python-s>
- **Tags:** nenhuma